

Introduction

- To identify visible parts of a scene from a chosen viewing position is major concern.
- Visible Surface Detection methods are also referred to as hidden-surface elimination methods
- Although there can be subtle differences between identifying visible surfaces and eliminating hidden surfaces. For example: For wireframe displays, we may not want to actually eliminate the hidden surfaces, but rather to display them with dashed boundaries or in some other way to retain information about their shape

Categories

Visible-surface detection algorithms are broadly classified into two categories as below:

1. Object-space methods

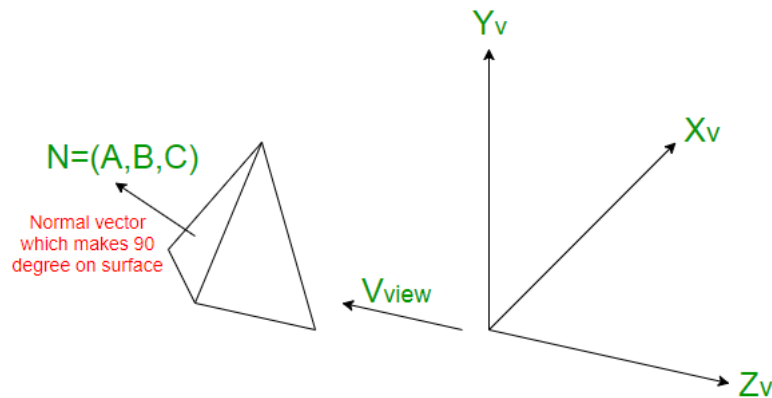
- ➔ Deals with object definition directly
- ➔ Compares objects and parts of objects to each other within the scene definition to determine which surfaces are visible.

2. Image-space methods

- ➔ Deals with Projected images
- ➔ visibility is decided point by point at each pixel position on the projection plane
- ➔ Most visible-surface algorithms use this approach

Back Face Detection (Plane Equation method)

A fast and simple object-space method used to remove hidden surface from a 3D object is called plane equation method. It is based on the "inside-outside" tests.



A point (x, y, z) is "inside" a polygon surface with plane parameters A , B , C , and D if

$$Ax + By + Cz + D < 0$$

When an inside point is along the line of sight to the surface, the polygon must be a back face. We can simplify this test by considering the normal vector $\mathbf{N}$ to a polygon surface which has Cartesian components (A, B, C) . If $\mathbf{V}$ is the vector in viewing direction from the eye (camera) position as shown in above figure, then this polygon is back-face if $\mathbf{V} \cdot \mathbf{N} > 0$

If Object description has been converted to projection co-ordinates and viewing direction is parallel to viewing Z_v axis, then

$$\mathbf{V} = (0, 0, V_z) \text{ and } \mathbf{V} \cdot \mathbf{N} = V_z \cdot C$$

Then we need to consider the sign of C , the Z -component of the Normal Vector $\mathbf{N}$.

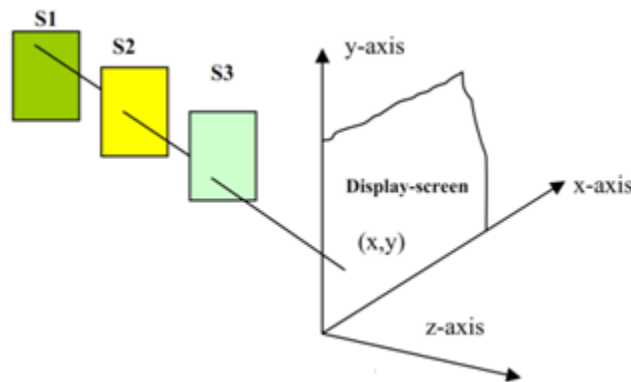
So any polygon face is back-face if its Normal Vector has Z -component value $C \leq 0$.

Depth Buffer Method (Z-buffer Method)

- ➔ A commonly used image-space approach to detecting visible surfaces is the depth-buffer method, which compares surface depths at each pixel position on the projection plane.
- ➔ Each surface of a scene is processed separately, one point at a time across the surface.
- ➔ The method is usually applied to scenes containing only polygon surfaces, because depth values can be computed very quickly and the method is easy to implement

- ➔ With object descriptions converted to projection coordinates, each (x, y, z) position on a polygon surface corresponds to the orthographic projection point (x, y) on the view plane.

Therefore, for each pixel position (x, y) on the view plane, object depths can be compared by comparing z values.



Z-buffer method need two buffer area. One buffer area store the depth information (z -value) and other store the intensity information (Refresh Buffer).

Algorithm

1. Initialize the depth buffer and refresh buffer so that for all buffer positions (x, y) ,
 $\text{Depth}(x, y) = 0$ and $\text{refresh}(x, y) = I_{\text{background}}$
2. For each position on each polygon surface, compare depth values to previously stored values in the depth buffer to determine visibility.
 - a) Calculate the depth z for each (x, y) position on the polygon.
 - b) If $z > \text{depth}(x, y)$, then set
 $\text{Depth}(x, y) = z$, $\text{refresh}(x, y) = I_{\text{surface}}(x, y)$

Where,

- ➔ $I_{\text{background}}$ = value for the background intensity
- ➔ $I_{\text{surface}}(x, y)$ = projected intensity value for the surface at pixel position (x, y) .
3. After processing all the surface, we will get visible surface in $\text{depth}(x, y)$ and intensity value in $\text{refresh}(x, y)$.

Depth - Calculation

Depth values for a surface position (x, y) are calculated from the plane equation for each surface:

$$Z = (-Ax - By - D) / C$$

Let z' be depth at position (x+1, y)

$$z' = (A(x+1) - By - D) / C$$

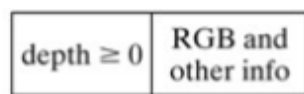
On simplifying,

$$z' = z - A/C$$

Where, $-A/C$ is constant for each surface so succeeding depth across (x+1) scanline can be obtained from preceding values using simple calculation.

A-Buffer Method

- An extension of the ideas in the depth-buffer method.
- A drawback of the depth-buffer method is that it can only find one visible surface at each pixel position.
- In A-Buffer Method more than one surface intensity can be taken into consideration at each pixel position.
- Each position in the A-buffer has two fields:
 - ➔ **depth field** - stores a positive or negative real number
 - ➔ **Intensity field** - stores surface-intensity information or a pointer value.



(a)



(b)

- a) If $\text{depth} \geq 0$, then surface data field stores the depth of that pixel position as before.
(When pixel overlap by only one surface)
- b) If $\text{depth} < 0$, then surface data field stores a pointer to a linked list of surface data
(When pixel overlap by only multiple surfaces)

Data for each surface in the linked list includes

- RGB intensity components
- opacity parameter (percent of transparency)
- depth
- percent of area coverage
- surface identifier
- other surface-rendering parameters
- pointer to next surface

Scan Line Method

An image space method for identifying visible surfaces

Computes and compares depth values along the various scan-lines for a scene.

Two important tables are maintained:

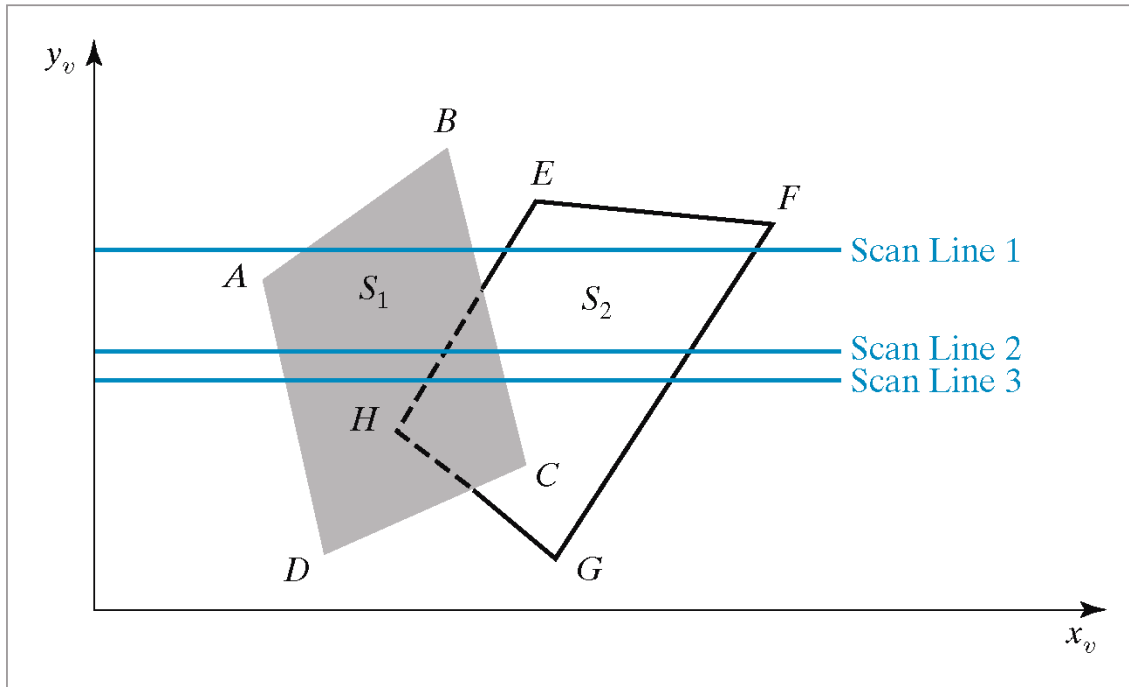
- The edge table
- The surface facet table

The edge table contains:

- Coordinate end points of each line in the scene
- The inverse slope of each line
- Pointers into the surface facet table to connect edges to surfaces

The surface facet table contains:

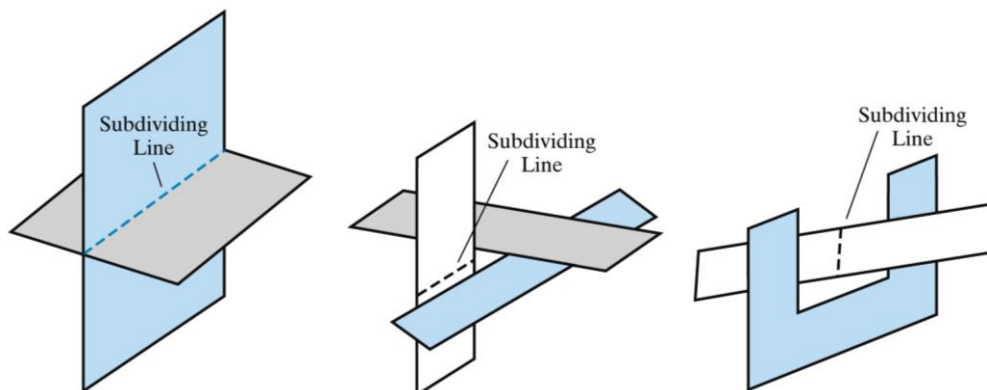
- The plane coefficients
- Surface material properties
- Other surface data
- Maybe pointers into the edge table



- In the figure above, active edge list for Scan Line 1 contains information from edge table for AB, BC, EH and FG.
- For Position along Scan Line 1 between AB & BC and between EH & FG flags for S_1 and S_2 are ON. As both flags are not ON at the same time, the depth calculation is not needed.
- For Scan Line 2 and 3, between edge EH and BC both flags S_1 and S_2 are ON. In this case, z-value is calculated and surface portion with highest z-value is visible.

Limitations of Scanline method

The scan-line method runs into trouble when surfaces cut through each other or otherwise cyclically overlap.



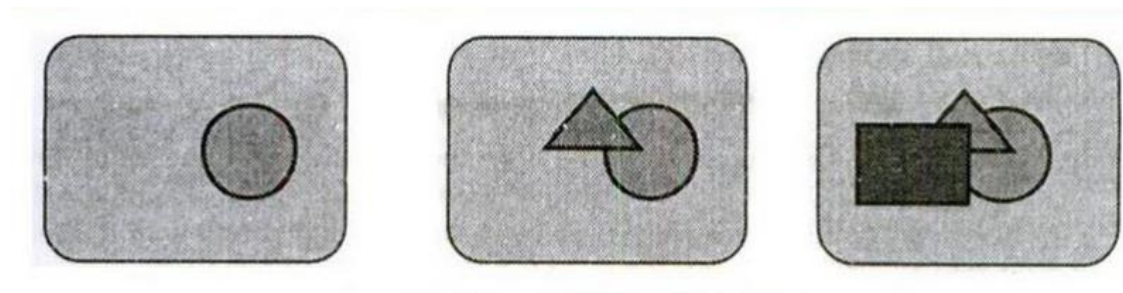
Depth Sorting Method (Painter's Algorithm)

Using both image-space and object-space operations, the depth-sorting method performs the following basic functions:

1. Surfaces are sorted in order of decreasing depth. (Object/Image space method)
2. Surfaces are scan converted in order, starting with the surface of greatest depth.
(Image space method)

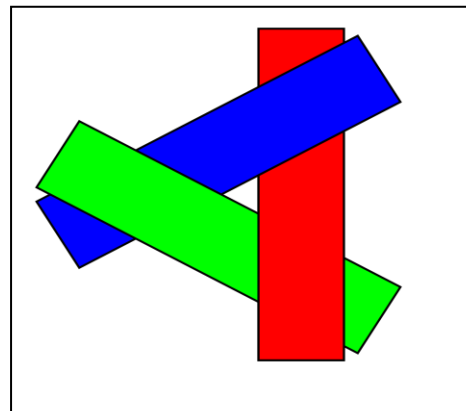
Algorithm Steps

1. Sort all polygon surface according to the smallest z-coordinate of each surface.
2. Resolve any ambiguity this may cause when splitting polygon.
3. Scan conversion each polygon in ascending order of smallest z-coordinate (polygon of greatest depth).



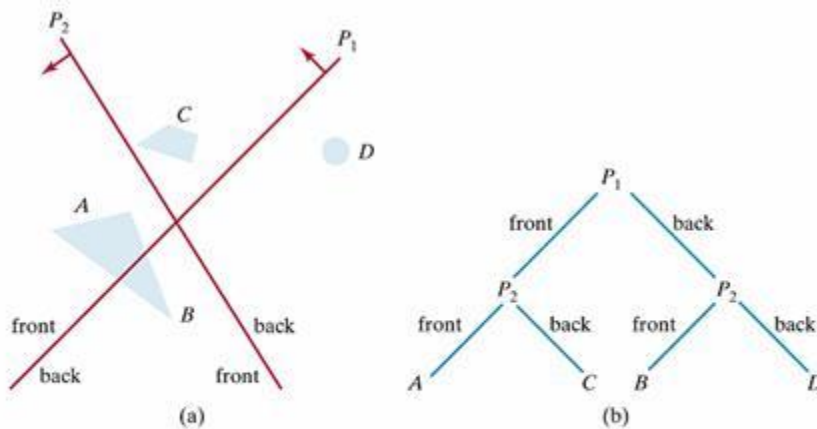
The newly displayed surface partly or completely obscured the previously displayed surface.

In some cases polygon may overlap each other (polygon may have same depth). Here polygon is split and scan converted separately. In the figure below polygon need to be split and scan converted separately.



Binary Space Partitioning (BSP) Method

A binary partitioning method is efficient method for determining object visibility by partitioning surface onto the screen from back to front by constructing a binary tree. In this method surface are divided by partitioning planes to determine the surface inside or outside relative to viewing direction. Multiple planes divides the polygon surfaces into back and front and a binary tree is constructed from them with plane as root and polygon surface as terminals.

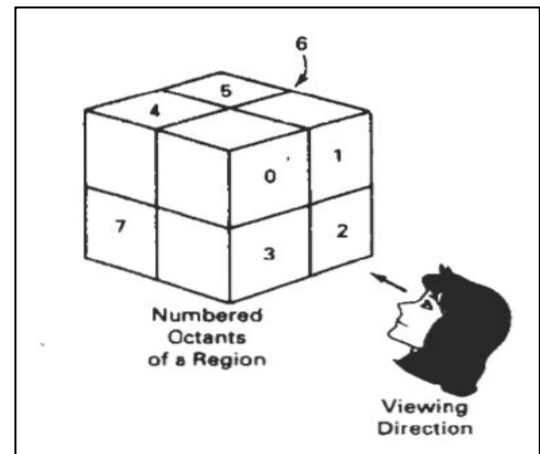


A region of space (a) is partitioned with two planes P_1 and P_2 to form the BSP tree representation shown in (b).

- Here, Plane P_1 partition the space into two sets of objects; one set is back and other set is front relative to viewing direction.
- Plain P_2 again intersects the surface and binary tree is constructed as in figure above. The object are displayed by traversing the tree in the back to front order.
- This method is useful for surface which remains constant but viewing direction changes from one direction to another.

Octree Method

In octree representation, hidden-surface elimination is accomplished by projecting octree nodes onto the viewing surface in a front-to-back order. In Fig. the front face of a region of space (the side toward the viewer) is formed with octants 0, 1, 2, and 3. Surfaces in the front of these octants are visible to the viewer. Any surfaces toward the rear of the front octants or in the back octants (4, 5, 6, and 7) may be hidden by the front surfaces.

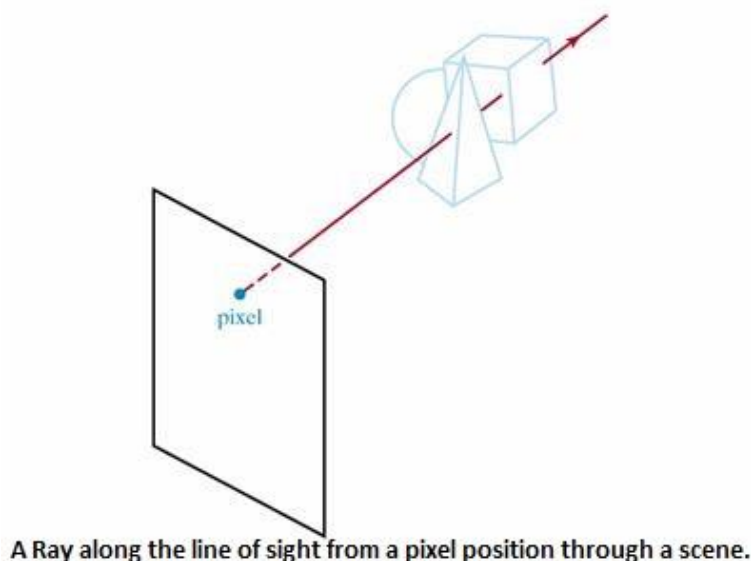


After octant sub-division and construction of octree, entire region is traversed using Depth First Method.

Ray Tracing (Ray Casting)

- The ray casting algorithm for hidden surfaces employs no special data structures.
- A ray is fired from the eye through each pixel on the screen in order to locate the polygon in the scene closest to the eye.
- The color and intensity of this polygon is displayed at the pixel.

Ray casting is easy to implement for polygonal models because the only calculation required is the intersection of a line with a plane.



Algorithm

1. Through each pixel, fire a ray to the eye.
2. Intersect the ray with each polygonal plane.
3. Reject intersections that lie outside the polygon.
4. Accept the closest remaining intersection i.e. the intersection with the smallest value of the parameter along the line.

The main advantage of the ray casting algorithm for hidden surfaces is that ray casting can be used even with non-polygonal surfaces.

The main disadvantage of ray casting is that the method is slow.